

Optimizing Dataplane OS through Intra-host Network Analysis

Abstract

As datacenter networks scale to multi-hundred gigabit speeds, understanding the *intra-host* network (*e.g.*, the data path between the NIC and CPU/memory) becomes increasingly critical. While significant attention has been given to optimizations of network hardware and system software (*e.g.* kernel bypassing OS), potential opportunities of OS optimizations according to complex interactions within the host’s I/O subsystem remain poorly understood.

This paper presents a systematic analysis of intra-host network bottlenecks in high-speed packet processing using DPDK. We dissect the Integrated I/O (IIO) stack, including the PCIe Root Complex, IOMMU (Intel VT-d), IOTLB, and Data Direct I/O (DDIO), to understand how these components interact and where performance bottlenecks arise. Through detailed measurements combining Intel PCM counters, NIC firmware telemetry, and DPDK profiling, we identify critical bottlenecks for I/O intensive workloads.

Our analysis provides actionable insights for optimizing CPU-based packet processing and lays the groundwork for understanding similar bottlenecks in more heterogeneous systems for ML workloads. As modern ML workloads increasingly rely on high-bandwidth communication between GPUs, NICs, and storage devices within a single host, the intra-host network analysis methodology we develop here becomes essential. Our future work aims to extend this characterization to GPU hosts, where PCIe/NVLink interconnects and GPU memory hierarchies introduce additional complexity to the intra-host data path.

1 Introduction

Reference Documents.

- **Intel Virtualization Technology for Directed I/O [8]:** Defines the hardware architecture and system software interface for Intel VT-d technology, which supports I/O device virtualization, such as DMA or interrupt remap-

ping (*i.e.*, hardware-level address/ID translation), etc., on Intel platforms.

- **Intel Data Direct I/O Technology (Intel DDIO): A Primer [5]:** Explains how Intel DDIO enables I/O devices to read from and write directly to the CPU’s Last Level Cache (LLC), bypassing main memory to reduce latency and improve system performance and power efficiency.
- **Intel Xeon Processor Scalable Memory Family Uncore Performance Monitoring [6]:** Defines the architecture, events, and control registers for Performance Monitoring Units (PMUs) in the uncore domain (memory controller, IIO, interconnect, etc.) of Intel Xeon Scalable processors.
- **Utilizing the Intel Xeon Processor Scalable Family IIO Performance Monitoring Events [7]:** Explains IIO (Integrated I/O) traffic flow in Intel Xeon Scalable processors and describes how to use performance monitoring events (PerfMon) to analyze I/O bandwidth and performance.
- **Intel Xeon Processor Datasheet Volume Two: Registers:** Documents uncore Configuration Space Registers (CSRs), IIO MMIO registers, and core MSRs (Machine Specific Registers).
 - Intel Xeon Processor Scalable Family (1st Gen Skylake-SP, 2nd Gen Cascade Lake) [4]
 - 3rd Gen Intel Xeon Scalable Processor (Ice Lake) [3]

2 background

Throughout this section, we use $\odot$ to denote logical/abstract concepts and $\oplus$ to denote physical hardware units.

2.1 IIO Stack

This section describes the Integrated I/O (IIO) stack based on [4, 7, 8]. Figure 1 illustrates the flow of a DMA request from

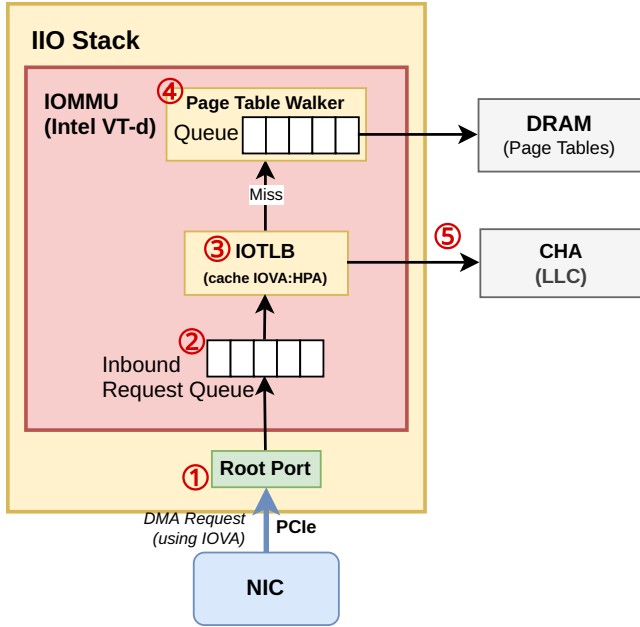


Figure 1: DMA request flow through the IIO stack. ① The NIC initiates a DMA request using an IOVA, which traverses the PCIe link to the Root Port inside the IIO. ② The request enters the Inbound Request Queue of IOMMU (VT-d engine). ③ The IOMMU checks the IOTLB for address translation (IOVA to HPA). ④ On a IOTLB miss, the request enters the Page Table Walker (PTW) Queue; the PTW then reads the page tables from DRAM (incurring latency overhead) and caches the result in the IOTLB. ⑤ On a hit (or after ④), the request reaches the target CHA determined by address hashing.

the NIC through the IIO stack. Figure 2 illustrates system architecture of our testbed nodes (node5 and node6).

2.1.1 PCIe Hierarchy.

The **Root Complex (RC)** is the logical root of the PCIe hierarchy (L), connecting the processor and system memory to the PCIe bus, and is physically implemented within the IIO stack (H). It translates system memory addresses for PCIe devices and vice versa, making PCIe devices appear local to the CPU. A **Root Port (RP)** (H) is a specific port within the Root Complex that creates a PCIe link to downstream devices (e.g., NICs, GPUs) or switches. The **Integrated I/O (IIO)** (H) is a block of circuitry on the CPU that integrates the Root Complex, Root Ports, DMA engines, and other I/O controllers into a single unit, streamlining CPU-to-I/O communication.

2.1.2 IIO as Gateway.

The IIO stack serves as a bridge between PCIe lanes and the processor’s internal mesh interconnect. For PCIe-based

network devices such as NICs, the IIO stack acts as a gateway that manages all traffic between the device and the host system. When a NIC reads data from main memory for transmission (DMA Read) or writes received data to memory (DMA Write), the data flow must pass through the IIO stack.

2.1.3 IOMMU[®] (Intel VT-d[™]).

Intel VT-d (Virtualization Technology for Directed I/O) is Intel’s architecture specification of I/O Memory Management Unit (IOMMU), integrated within the IIO stack alongside the PCIe root complex. The IOMMU intercepts DMA requests from I/O devices before they reach main memory, providing address translation and memory protection. When a device performs DMA, it uses I/O Virtual Addresses (IOVAs) rather than physical addresses. The IOMMU translates these IOVAs to Host Physical Addresses (HPAs) using multi-level page tables stored in DRAM, similar to CPU virtual memory. This DMA remapping enables devices to access non-contiguous physical memory as if it were contiguous, simplifying buffer management, while also providing memory isolation: each device (or group of devices) can have its own address space, preventing errant or malicious devices from accessing unauthorized memory regions. **info:** The IOMMU can be enabled or disabled via BIOS (Intel VT-d option) or kernel boot parameters (`intel_iommu=on/off`, `iommu=pt`). When IOMMU is disabled, devices issue physical addresses directly—no address translation occurs. In DPDK, this corresponds to `iova-as-pa` mode, where the application maps hugepage virtual addresses to physical addresses via `/proc/self/pagemap` and passes them directly to the NIC. When IOMMU is enabled, DPDK can use `iova-as-va` mode, where IOVAs are virtual addresses and the IOMMU handles VA→PA translation. Disabling IOMMU eliminates IOTLB miss penalties and page table walk overhead, yielding more deterministic performance—hence most bare-metal DPDK benchmarks disable it. However, IOMMU is essential for DMA memory isolation (security) and device passthrough in virtualized environments (e.g., SR-IOV).

2.1.4 IOTLB[®].

Since page tables reside in DRAM, each address translation would require multiple memory accesses to walk the page table hierarchy. To avoid this overhead, the IOMMU caches recent translations in the I/O Translation Lookaside Buffer (IOTLB), a SRAM structure inside the IOMMU. When a DMA request arrives, the IOMMU first checks the IOTLB. On a hit, the translation is returned immediately and the DMA proceeds.

2.1.5 Page Table Walker[®].

On an IOTLB miss, the request is queued to the Page Table Walker (PTW) circuit block, which traverses the multi-level

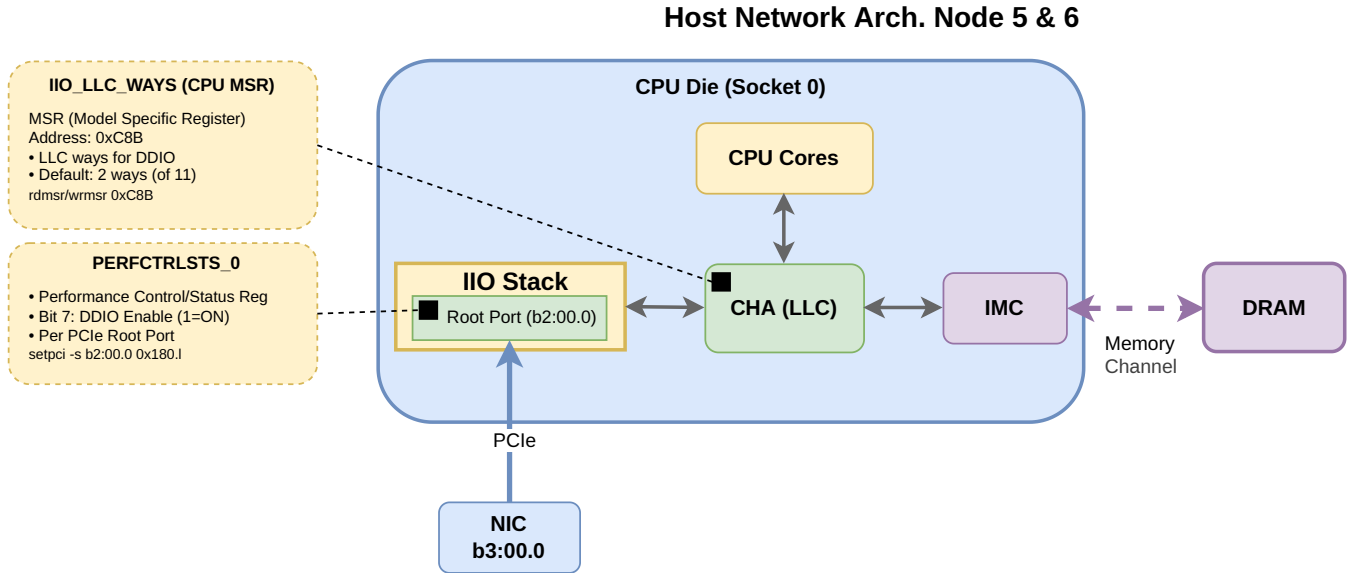


Figure 2: Node 5/6 architecture.

page table in DRAM to resolve the translation. Once complete, the result is cached in the IOTLB and the DMA request proceeds. High IOTLB miss rates can significantly degrade I/O performance, as each miss incurs multiple DRAM accesses and queueing delays in the Page Table Walker.

2.2 Intel DDIO (Data Direct I/O)

This section summarizes Intel’s Data Direct I/O technology based on [5].

Traditional I/O architectures were designed assuming slow networking speeds and small caches, treating main memory as the primary destination and source for I/O data rather than the scarce cache. This forced frequent trips to memory for I/O data transfers, degrading system performance and increasing power consumption. Intel DDIO was introduced to leverage faster networking and larger caches more efficiently, enabling NICs to communicate directly with the host processor’s LLC and drastically reducing memory access. As a result, DDIO increases system I/O bandwidth, reduces latency, and lowers power consumption, increasing overall system performance.

2.2.1 DMA Write (RX Path).

When a NIC receives a packet, it initiates DMA write operations to transfer the packet data and control structures (*e.g.*, descriptors) to host memory for protocol processing by the CPU. **Without DDIO:** Data is written directly to memory, invalidating any cached copy; when the CPU later reads the data for processing, it incurs a cache miss and must fetch from memory. **With DDIO:** Data is delivered directly to the LLC via *Write Update* (in-place update if address is already

cached) or *Write Allocate* (new allocation if not cached), so subsequent CPU reads hit the cache. Write Allocate is limited to 10% of LLC capacity to prevent cache pollution from high-rate streams that exceed CPU processing speed; this is not reserved space—it remains fully available to CPU applications.

2.2.2 DMA Read (TX Path).

When a NIC transmits a packet, it initiates DMA read operations to fetch packet data and control structures (*e.g.*, descriptors) from host memory. The CPU first creates the packet data in its cache hierarchy, then notifies the NIC to begin transmission. **Without DDIO:** Each NIC read causes the data to be evicted from cache, so when the CPU later accesses the buffer (*e.g.*, to free it), it must fetch the data from memory again. **With DDIO:** The NIC read is directly served from cache *without eviction*, keeping the data available for software reuse and achieving zero memory trips in the ideal case. Note that if data is not found in cache, it is fetched from memory but *not allocated* into the cache hierarchy (unlike DMA writes).

2.2.3 Enabling/Disabling DDIO Write.

DDIO Write can be controlled through the `Use_Allocating_Flow_Wr` bit (bit 7) of `PERFCTRLSTS_0` register (offset: 180h) in the PCIe Root Port [4]. For example, the register value on node6 can be checked with `$ setpci -s b2:00.0 0x180.1`. This register bit controls whether DMA writes use allocating flow to LLC (with DDIO) or non-allocating flows to DRAM (legacy mode without DDIO). In the non-allocating mode, if the data being delivered

happens to be in the CPU’s caching hierarchy, it is invalidated [5]. Importantly, this bit only affects write operations: setting it to 0 disables LLC allocation for writes, but read operations continue to use the cache-efficient DDIO behavior (no eviction on LLC hit). The “legacy mode” described in [5] (*i.e.*, eviction after read) refers to older architectures without DDIO, not the state when this register bit is unset. In other words, the DDIO disable switch is a half-switch that only controls write allocation; read-side benefits remain active as long as the processor supports DDIO.

2.2.4 Tuning LLC Occupancy.

Although [5] mentions that Write Allocate operations are restricted to 10% of the LLC and its fixed (cannot be changed), [2] (in Section 4.4) mentions that it can be adjusted via the IIO LLC WAYS MSR with the address of 0xC8B. The behavior may vary across different CPU architectures [9]. This needs to be further investigated.

2.2.5 CHA Processing Flow.

After the DMA request arrives at the target CHA (Caching Home Agent), the CHA performs coherency checks and cache operations differently depending on whether it is a DMA Write (RX) or DMA Read (TX). Figure 3 illustrates both flows.

3 Understanding DPDK

Assumptions. For simplicity of exposition, throughout this section we assume: (i) the NIC port in use is on the same NUMA node as the application’s lcores (NUMA-local), and this mapping is fixed across experiments; and (ii) each lcore is assigned exactly one dedicated RX queue and one dedicated TX queue (no queue sharing across cores, no multi-queue per core). These assumptions remove confounding effects from NUMA remoteness and queue-to-core contention, allowing us to reason about a single lcore’s TX/RX pipeline in isolation. Relaxing these assumptions (NIC-remote NUMA placement, multiple queues per core, shared queues across cores) introduces additional effects—NIC MTT cache pressure, cross-socket DMA over UPI, RSS imbalance, DDIO scope loss—whose characterization we leave to future work.

This section presents a top-down view of the packet transmit (TX) and receive (RX) paths in DPDK, from the user-space application down to the NIC hardware. Unlike the Linux kernel network stack—where packets traverse multiple protected subsystems (socket, TCP/IP, netdevice, driver) with interrupt-driven, per-packet processing—DPDK bypasses the kernel entirely and relies on user-space polling, zero-copy buffers, and direct doorbell-based interaction with the NIC. Understanding every stage of this pipeline is a prerequisite

Table 1: DPDK pipeline components and their responsibilities. Layers are ordered top-down from application to NIC hardware.

Layer	Responsibility
Application	Poll loop; mbuf construction (TX) or consumption (RX); packet classification/forwarding logic.
ethdev API	Generic NIC abstraction: rte_eth_tx_burst() / rte_eth_rx_burst(), queue configuration, offload negotiation.
mempool / mbuf	Lockless per-core cached buffer pool; pre-allocated DMA-capable huge-page memory; rte_mbuf metadata and data payload.
PMD (mlx5)	Vendor driver: translates mbuf bursts into NIC work requests (WQEs) and consumes completions (CQEs); manages SQ/RQ/CQ rings; rings the doorbell.
NIC HW (SQ/RQ)	Send / Receive Queue: ring of descriptors (WQEs) in host memory.
NIC HW (CQ)	Completion Queue: NIC writes CQEs back to host memory after each WQE is processed.
NIC HW (engine)	WQE fetch, LSO/inline, scatter-gather DMA engines, MAC/PHY; packet classification and RSS steering on RX.
IIO / DDIO	PCIe Root Complex routes DMAs; DDIO directs DMA writes to LLC and serves DMA reads from LLC without eviction (see §2).

for reasoning about the bottlenecks and tuning opportunities we analyze in §5.

Figure 4 presents the end-to-end pipeline; Table 1 lists the responsibilities of each layer.

3.1 Sender-side (TX) Pipeline

The TX path begins with the application running a polling loop on a dedicated lcore. Packets already exist as `rte_mbuf` structures in a pre-allocated mempool; there is no `write` system call, no socket buffer copy, and no kernel transition.

`rte_eth_tx_burst()`. The application invokes this ethdev API with a burst of mbuf pointers. The call is a thin inline wrapper that dispatches directly to the PMD’s per-queue TX function—no locking, no indirect syscall, typically only a function pointer dereference.

PMD (mlx5) TX processing. The mlx5 PMD consumes the mbuf burst and produces a sequence of *Work Queue Entries* (WQEs) in the Send Queue (SQ), a ring buffer in host huge-

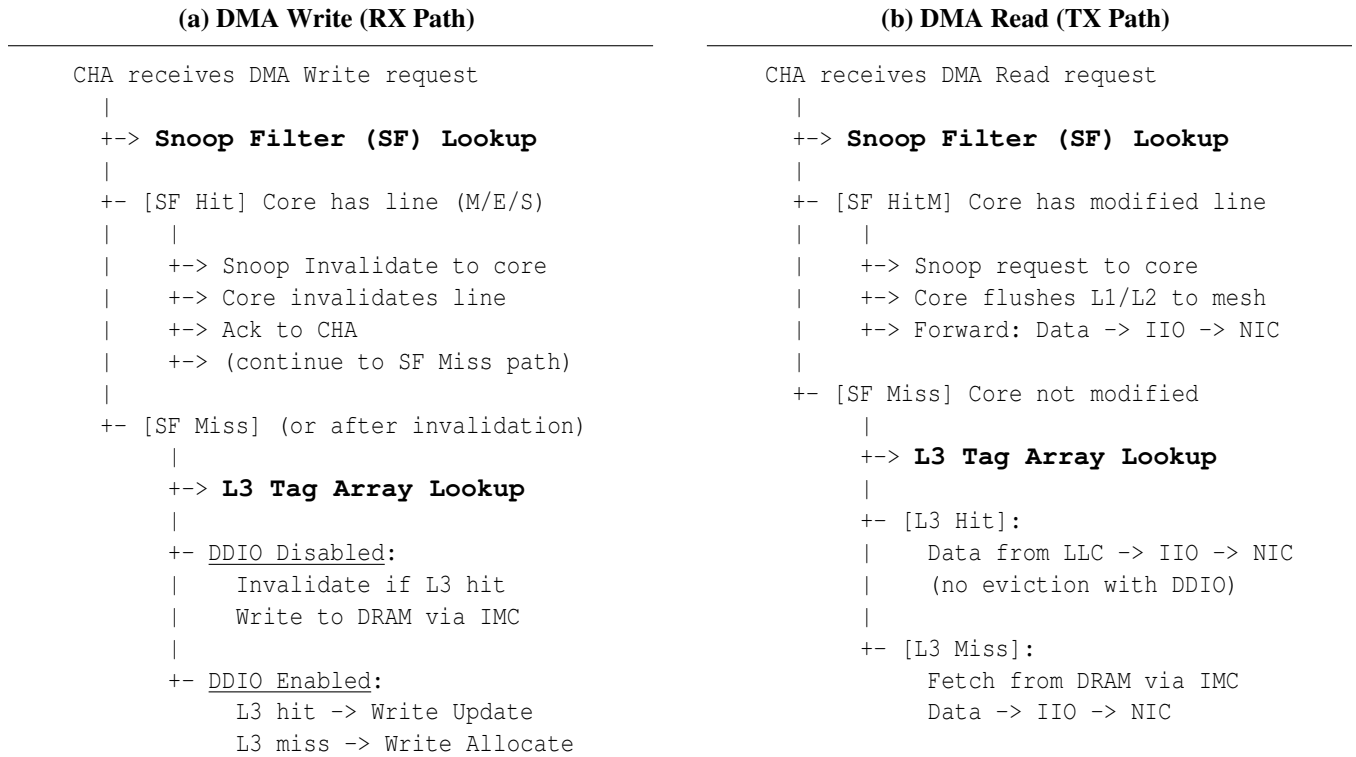


Figure 3: CHA processing flow for DMA Write (RX path) and DMA Read (TX path). Both paths begin with a Snoop Filter lookup for cache coherency, followed by L3 tag array lookup. DDIO affects the write allocation policy (a) and preserves cache contents on reads (b).

page memory. Three features of WQE construction are critical for performance:

- **Inline data:** for small packets, the packet payload (or its first `txq_inline_mpw` bytes) is copied directly into the WQE itself. This saves the NIC a subsequent DMA Read of the payload, trading a CPU-side copy for one less PCIe round trip.
- **Multi-Packet Write (MPW):** multiple small packets are encoded into a single WQE, amortizing WQE overhead across a batch.
- **LSO (Large Send Offload):** for large packets, a single WQE describes a TCP segment that the NIC hardware segments into multiple MTU-sized frames, avoiding per-segment CPU cost.

After writing the WQEs, the PMD updates the producer index and *rings the doorbell*—a single MMIO write to the NIC’s BAR that notifies the NIC of new work. Doorbell writes may be batched across the burst to reduce PCIe transactions.

NIC hardware (TX engine). Upon seeing a new doorbell value, the NIC’s TX scheduler issues a PCIe *DMA Read* for the WQE. If the WQE references external mbuf data (non-inline), a second DMA Read fetches the payload. Both reads traverse the IIO stack, where the IOMMU performs IOVA→HPA translation (unless IOMMU is disabled) and

DDIO may serve the data directly from LLC without eviction (§2). The TX engine then hands the frame to the MAC/PHY for transmission on the wire.

Completion. After transmission, the NIC writes a *Completion Queue Entry* (CQE) back to host memory via DMA Write. The PMD polls the CQ on subsequent `rte_eth_tx_burst()` invocations (or during TX cleanup) to reclaim mbufs and free SQ slots for new work. No interrupt is delivered in the steady-state fast path.

3.2 Receiver-side (RX) Pipeline

The RX path is almost symmetric to TX, but the NIC drives the initial activity.

NIC hardware (RX engine). A frame arrives on the wire and is parsed by the NIC’s classifier. *Receive-Side Scaling* (RSS) hashes the 4-tuple (or configurable fields) and selects one of several Receive Queues (RQs) to steer the packet to a specific core. The NIC then pops an RQ descriptor—pre-posted by the PMD and pointing to a free mbuf’s data buffer—and issues a PCIe *DMA Write* of the frame payload into that buffer. With DDIO enabled, this write lands in the LLC (Write Allocate or Write Update), keeping the data cache-resident

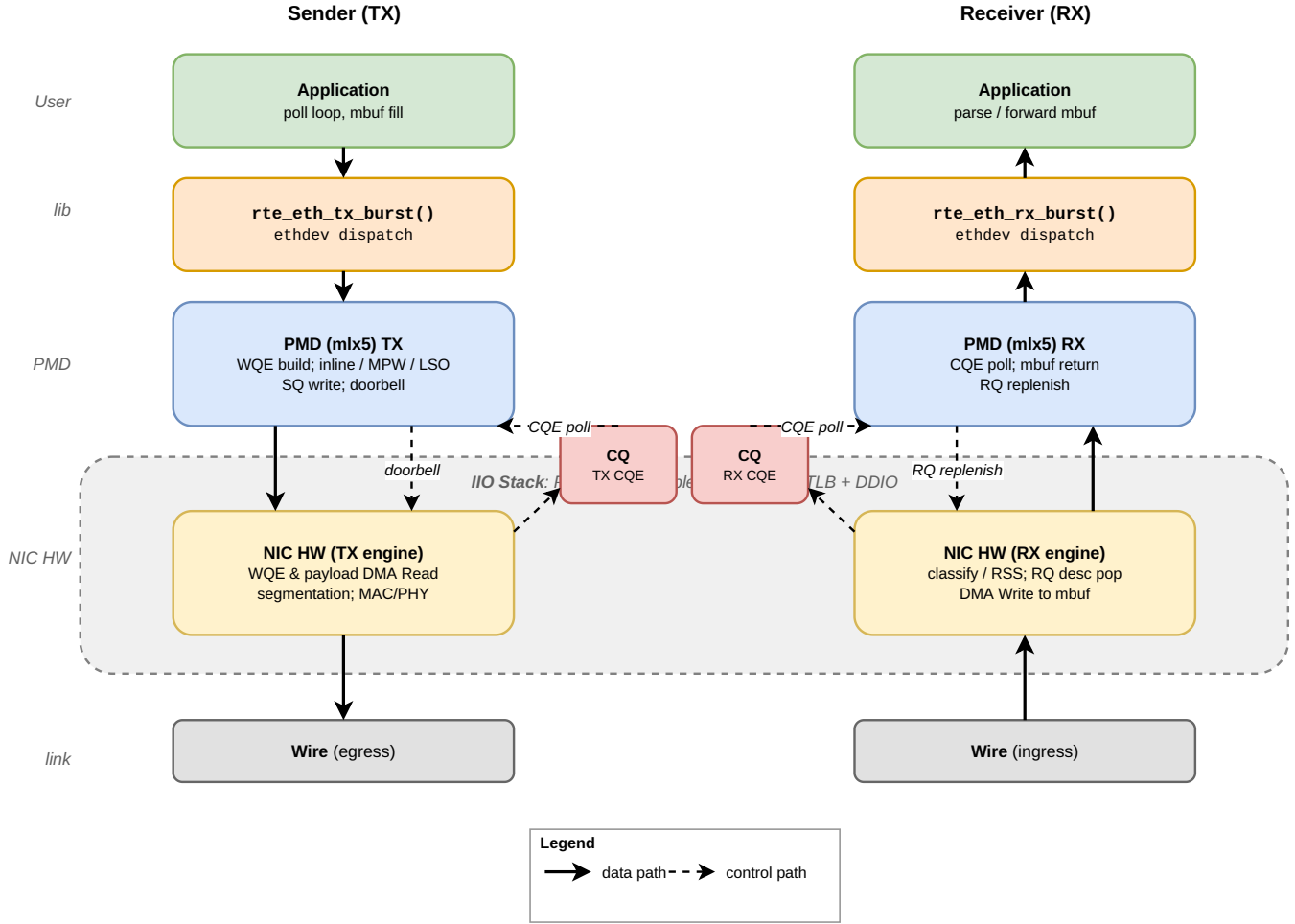


Figure 4: End-to-end DPDK packet processing pipeline on a Mellanox ConnectX NIC. **Left:** TX (sender) path. **Right:** RX (receiver) path. Solid arrows denote the data path; dashed arrows denote the control path (doorbells, completion notifications). All software stages run in a single user-space poll loop on a dedicated lcore; no kernel transition or interrupt occurs after initialization.

for the CPU. The NIC finally writes a CQE describing the completed reception.

PMD (mlx5) RX processing. `rte_eth_rx_burst()` polls the CQ: it reads CQEs (hopefully from LLC, thanks to DDIO), extracts the per-packet metadata (length, RSS hash, checksum status, offloads), and hands the corresponding mbufs back to the application. The PMD immediately replenishes the RQ with fresh mbufs from the mempool so the NIC never starves. No IRQ, no softirq, no GRO—the equivalent of batching is done either in software by the application or by NIC-side LRO if enabled.

Application processing. The application receives the mbuf burst directly from the poll call. Because the mbuf payload was DMA'd into LLC, the first cache-line access is typically a hit; subsequent protocol parsing or forwarding (as in L3FWD) proceeds without touching DRAM in the ideal case. After

processing, the application either re-queues the mbuf for TX (forwarding) or frees it back to the mempool.

3.3 Key Differences vs. the Linux Kernel Stack

Compared to the Linux stack characterized by [1] (reproduced conceptually in their Figure 1), DPDK eliminates or replaces several layers:

- **No socket/TCP/IP by default:** DPDK delivers raw Ethernet frames; any protocol logic is library- or application-level.
- **No skb management:** the `rte_mbuf` is allocated once from a pre-sized mempool and recycled; there is no equivalent to the kernel's per-skb alloc/free or page-pool refill overhead.
- **No IRQ / softirq / NAPI:** a single poll loop on a pinned lcore replaces interrupt-driven processing, eliminating context switches and scheduling overhead at the cost of 100% CPU utilization on the poll core.

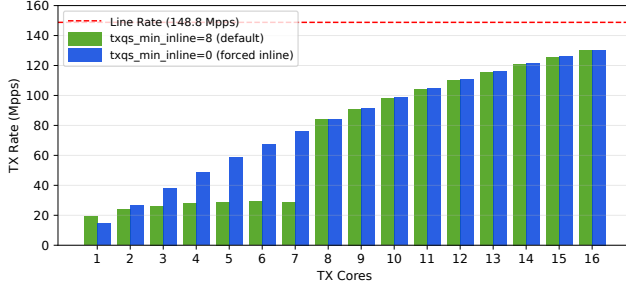


Figure 5: PKTGEN TX rate scaling, 1–16 TX cores on node5, comparing the ConnectX-5 default (`txqs_min_inline=8`, inline OFF until 8 queues) with forced inline mode (`txqs_min_inline=0`, inline ON throughout). The two curves overlap exactly for 8+ cores; the entire 7→8 cliff in the default is the threshold being crossed, not a CPU-side scaling change.

- **No kernel/user data copy:** mbufs live in huge pages that are DMA-mapped once at startup; the NIC DMA’s directly into and out of the application’s address space.
- **No GRO/GSO software aggregation:** batching is explicit (burst size) and offloads (LSO, LRO, MPW) are exposed directly to the application.

The consequence is that the bottlenecks analyzed in [1]—data copy, skb allocation, scheduling, and lock contention—are largely *absent* in DPDK. Instead, DPDK performance is shaped by a different set of costs: per-WQE construction overhead in the PMD, PCIe round-trip latency for DMAs and doorbells, IOTLB pressure in the IIO stack, LLC contention via DDIO, and mempool cache locality. We analyze these in §5.

4 Performance Monitoring Tools

pcm-pcie The `pcm-pcie` tool reports coherence lookup results at the CHA (Caching and Home Agent) for PCIe transactions: a *hit* indicates the target address already exists in LLC and requires coherence action, while a *miss* indicates the address is not in LLC and requires no coherence action. Note that it has nothing to do with DDIO hit/miss. To verify the impact of DDIO ON/OFF, we use `pcm-memory` to measure DRAM bandwidth: DDIO ON results in low DRAM write, while DDIO OFF causes high DRAM write.

5 Evaluation

Setup. All experiments use 64-byte packets and default DPDK configuration parameters unless otherwise noted.

Table 2: TX scaling metrics: 1 core vs 2 cores.

Metric	1 Core	2 Cores	Ratio
<i>TX Rate</i>			
TX Rate (Mpps)	19.05	23.90	1.26×
<i>pcm-iio (IIO Stack)</i>			
IB Read (GB/s)	1.48	1.85	1.25×
IOTLB Hit (M/s)	20.30	25.32	1.25×
IOTLB Miss (M/s)	11.25	14.13	1.26×
IOTLB Miss Rate	35.65%	35.82%	≈same
<i>NeoHost (NIC Internal)</i>			
PCIe Inbound BW (Gb/s)	15.99	19.99	+25%
PCIe Avg Latency (ns)	1627	2468	+52%
PCIe Stall (No Compl.)	0	0	–
MTT L0 Miss Rate	4.56%	6.70%	+47%

5.1 TX Scaling Bottleneck Analysis

We analyze why TX throughput does not scale linearly with the number of cores without inline mode. Measurements were collected using: (1) `pcm-iio` for IIO stack counters (outstanding reads, IOTLB hit/miss), (2) NIC firmware parameters (`LOG_MAX_OUTSTANDING_WQE`, currently 128), and (3) IOMMU disable tests to isolate IOTLB overhead.

1 Core vs 2 Cores Comparison. Table 2 summarizes the key metrics. With 2 cores, TX rate improves by only 1.26× (not 2×), while PCIe latency increases by 52%.

Little’s Law Analysis. Using Little’s Law (Throughput = Outstanding Requests / Latency):

- **1 Core:** $(15.99 \text{ Gb/s} \times 1627 \text{ ns}) / (64 \text{ B} \times 8) \approx 51$ outstanding reads
- **2 Cores:** $(19.99 \text{ Gb/s} \times 2468 \text{ ns}) / (64 \text{ B} \times 8) \approx 96$ outstanding reads

While 2 cores issue $\sim 2\times$ outstanding requests, latency also increases proportionally, limiting throughput gains.

Key Findings.

1. **Not PCIe tag saturation:** PCIe Stall (No Completion) = 0, well below 256-tag limit.
2. **Bottleneck is PCIe latency increase:** 52% latency growth from 1→2 cores.
3. **IOTLB miss rate unchanged ($\sim 35.6\%$):** per-request miss rate is core-independent, but total misses increase with request volume, causing IIO stack queueing delays.
4. **NIC-side MTT cache contention:** MTT L0 miss rate increases 47% (4.56%→6.70%).

Q: Why does IOTLB miss rate stay the same ($\sim 35\%$) with more cores? IOTLB caches *page-level* translations

(IOVA→HPA), not per-packet addresses. With DPDK’s 2MB huge pages, one IOTLB entry covers thousands of 64B packets. Multiple cores sharing the same mempool access the same huge pages, so the working set of unique pages remains similar regardless of core count. The miss *rate* stays constant, but **total misses increase** (11.25→14.13 M/s), and these additional page table walks cause IIO stack queueing delays.

Conclusion. The shared bottleneck is the **IIO Stack** (PCIe Root Complex), not PCIe tags or IOTLB miss rate per se. The 35% IOTLB miss rate causes each DMA read to incur additional memory accesses for page table walks, creating queueing delays in the IIO stack that worsen as core count increases. Figure 6 shows the key metrics for 1, 2, 4, and 8 TX cores, and Figure 7 shows the full set of metrics.

5.2 Impact of Forced Inline Mode (txqs_min_inline=0)

The default ConnectX-5 setting txqs_min_inline=8 causes the mlx5 PMD to enable eMPW inline only once the application opens at least 8 TX queues. The discontinuity at 8 cores in Figure 5 (~28→84 Mpps, ~3×) is exactly this threshold being crossed—1–7 core runs use the non-inline path (NIC DMA-reads each payload), and 8+ core runs use the inline path (PMD copies the payload into the WQE, eliminating one PCIe round-trip per packet).

To isolate the contribution of inline mode itself, we re-ran the full 1–16 core TX-scaling sweep with txqs_min_inline=0, which forces the inline path for every core count. The result is overlaid on Figure 5; all other parameters are identical to the default-configuration run.

Table 3 reports the 1-core vs. 2-core breakdown for txqs_min_inline=0 in the same format as Table 2, so the two cases can be compared metric-by-metric.

Key observations.

1. **Single-core throughput drops ~24%** (19.0→14.6 Mpps). At one core the NIC has ample PCIe headroom (only ~15.9 Gb/s inbound under default), so saving a PCIe payload-read does not buy throughput, while the extra CPU-side WQE copy directly costs cycles in the poll loop. Inline is a net loss in this regime.
2. **2–7 cores scale near-linearly** (14.6→76.3 Mpps, ~5.2× over 6× cores) under forced inline, while the default flattens at ~28 Mpps from 3 cores onward. Table 3 shows why: with inline on, IOTLB miss rate stays at 2.75–7.34% (vs. 35% under default), MTT L0 miss rate is ≈0%, and PCIe avg. latency grows only +7% from 1 to 2 cores (vs. +52% under default). Eliminating the second per-packet DMA-read collapses the IIO-stack queueing that bottlenecked the default.

Table 3: TX scaling metrics with txqs_min_inline=0 (forced inline): 1 core vs. 2 cores. Same format as Table 2, enabling direct comparison.

Metric	1 Core	2 Cores	Ratio
<i>TX Rate</i>			
TX Rate (Mpps)	14.59	26.66	1.83×
<i>pcm-iio (IIO Stack)</i>			
IB Read (GB/s)	1.02	1.86	1.82×
IOTLB Hit (M/s)	2.00	3.65	1.82×
IOTLB Miss (M/s)	0.06	0.29	4.83×
IOTLB Miss Rate	2.75%	7.34%	+167%
<i>NeoHost (NIC Internal)</i>			
PCIe Inbound BW (Gb/s)	9.15	16.66	+82%
PCIe Avg Latency (ns)	372	398	+7%
PCIe Stall (No Compl.)	0	0	–
MTT L0 Miss Rate	0.00%	0.00%	–

3. **8+ cores: the two configurations are identical** (84.4 vs. 83.9 Mpps at 8 cores; matching within <0.5 Mpps for 9–16 cores). Once the default crosses the txqs_min_inline=8 threshold, both runs activate the same mlx5 inline path with the same txq_inline_mpw layout, so all downstream metrics (PCIe BW, latency, IOTLB/MTT miss) match. Figure 5 shows the two curves overlap exactly from 8 cores on.
4. **The 7→8 “jump” is purely a threshold artifact.** With inline forced on, the curve is monotonic and smooth (76.3→84.4 Mpps from 7 to 8 cores); the ~3× leap visible in Figure 5 is entirely caused by the default crossing into the same regime that =0 is in throughout, not by any inherent CPU-side scaling change.

Takeaway. txqs_min_inline is a non-monotone, regime-dependent knob. At 1 core the default (=8) wins by ~30%; at 2–7 cores forcing =0 wins by 1.1–2.7×; at 8+ cores the two are indistinguishable. No single static value is best across the operating range, motivating an adaptive policy that picks the inline threshold based on observed load (number of busy cores, PCIe bandwidth headroom) rather than a fixed device-driver default.

5.3 DDIO Impact Analysis

Figure 8 shows the impact of enabling/disabling Use_Allocating_Flow_Wr bit (as discussed in §2.2.3) on both PKTGEN (TX) and L3FWD (RX) nodes.

Key observations:

- pcm-pcie results (PCIe Read/Write) are not affected.

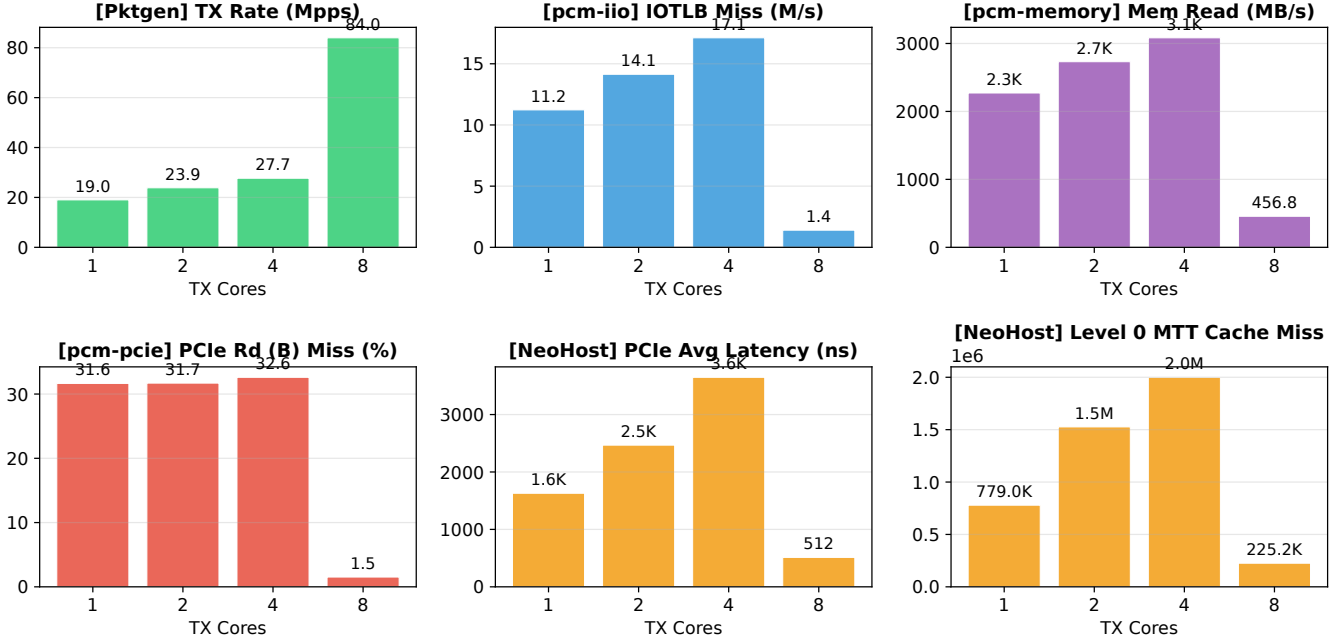


Figure 6: Key TX core scaling metrics (1, 2, 4, 8 cores): TX Rate, IOTLB Miss, Memory Read, PCIe Read Miss, PCIe Latency, and MTT Cache Miss.

- On PKTGEN (Pkt TX, DDIO Read), Both DRAM Read and Write are not affected.
- On L3FWD (Pkt RX, DDIO Write), Both DRAM Read and Write are significantly increased when DDIO Write is off, confirming that incoming packets bypass LLC and go directly to DRAM.

5.4 Impact of TX Burst Size (DEFAULT_PKT_TX_BURST)

DEFAULT_PKT_TX_BURST sets the number of packets the application passes to each `rte_eth_tx_burst()` call. In Pktgen this is a per-port runtime field (`pinfo->tx_burst`), defaulting to 32 and tunable at runtime through the Lua API (`pktgen.set(port, "burst", N)`). Because it sits at the application layer rather than inside the PMD, the same Pktgen binary can sweep the entire range [1, 256] without rebuilding—one of the few burst-related knobs that is truly runtime-tunable on the TX path.

We sweep `tx_burst` $\in \{1, 4, 8, 16, 32, 64, 128, 256\}$ across three TX core counts (1, 4, 16). All other parameters are kept at defaults: 64-byte packets, 1024 TX descriptors, `mlx5 txqs_min_inline=8` (CX-5 default), L3FWD in DROP mode to isolate the PKTGEN-side TX path. Figure 9 plots the resulting TX rate; Table 4 lists all 24 measured points.

Key observations.

- 1- and 4-core curves plateau by burst = 4. Beyond burst = 4 they vary by less than 5% all the way to burst = 256

Table 4: PKTGEN TX rate (Mpps) sweep over DEFAULT_PKT_TX_BURST. 64 B packets, L3FWD DROP mode, `txqs_min_inline=8` (mlx5 default).

Burst	1 core	4 cores	16 cores
1	6.95	9.03	10.11
4	18.17	27.76	39.80
8	18.89	29.08	78.42
16	19.66	28.45	126.29
32	19.16	27.90	130.33
64	19.40	28.01	132.01
128	19.60	27.56	131.11
256	19.43	27.60	131.13

(1 core: 18.2→19.4; 4 cores: 27.8→27.6). At low core counts the bottleneck is *not* per-call amortization—these workloads are CPU-bound on a single (or few) lcore, and once the per-call overhead is amortized over ~4 packets, larger bursts buy nothing.

- 16 cores shows a dramatic $\sim 3.2\times$ jump from burst = 8 to burst = 16 (78.4→126.3 Mpps). This is exactly the `MLX5_EMPW_MIN_PACKETS=14` threshold being crossed: below 14 packets per burst, the `mlx5` PMD cannot fill an eMPW WQE and falls back to single-packet sends; at or above 14, eMPW kicks in and the per-packet PCIe transaction count drops sharply (cf. Table 3).
- Beyond burst = 16, all curves are flat. 16 cores stays at 130–132 Mpps, indicating the link/I/O ceiling is reached

TX Core Scaling - All Metrics (1, 2, 4, 8 cores)

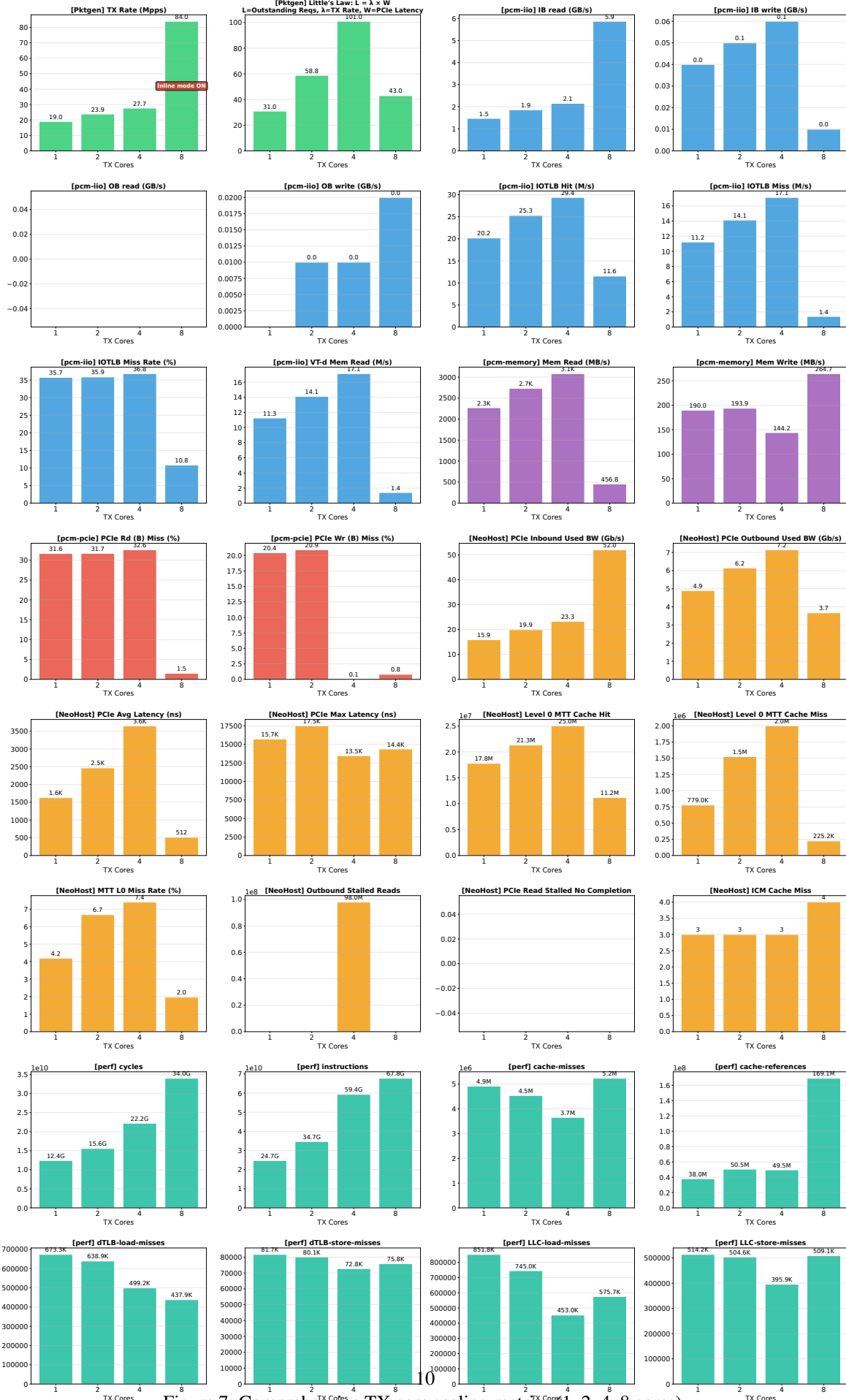


Figure 7: Comprehensive TX core scaling metrics (1, 2, 4, 8 cores).

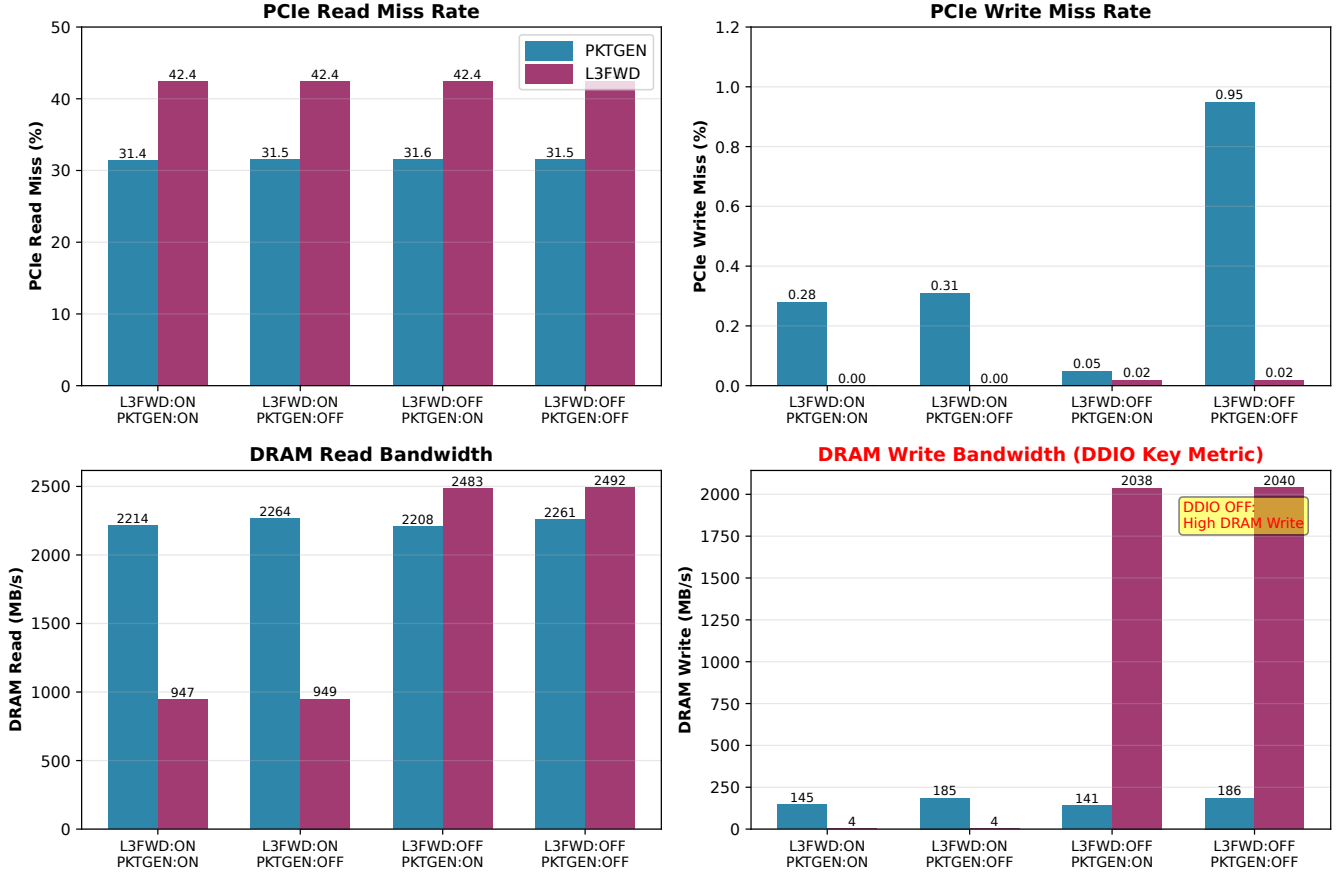


Figure 8: DDIO ON/OFF impact on pcm-pcie and pcm-memory metrics. DRAM Write bandwidth is the key indicator: when DDIO Write is OFF on the RX side (L3FWD), incoming packets bypass LLC and write directly to DRAM. PKTGEN: 1 TX core, 19 Mpps. L3FWD: 1 RX/TX core, 16 Mpps.

and additional batching gives no further gain.

4. **The optimal burst depends on the operating point.** At 1 core, burst = 8 is already within 1% of the best value; at 4 cores, burst = 8 is similarly optimal; at 16 cores, burst ≥ 16 is required to reach line-rate-region throughput. The Pktgen default of 32 is therefore over-sized for low-core workloads (no harm, but no benefit) and well-tuned for 16-core line-rate runs.
5. **Very small bursts (= 1) are uniformly catastrophic.** Even 16 cores reach only 10 Mpps when burst = 1, because every `rte_eth_tx_burst()` call pays full per-call overhead per packet and the eMPW path is entirely disabled.

Rate-limited probe: latency dimension. The Figure 9 sweep used a 100%-rate offered load on every burst setting, which conflates “does burst N matter?” with “can pktgen drive line rate at burst N ?”. To separate these, we re-ran the burst sweep at 16 cores while varying the offered TX rate via Pktgen’s rate setting (25%, 50%, 100% of line rate). Figure 10

(top) plots the resulting TX rate. Two findings stand out:

1. **The minimum-burst-to-saturate depends on offered rate.** At 25% rate, burst = 4 already saturates the target (37.6 Mpps); at 50% rate, burst = 8 saturates (75.1 Mpps); at 100% rate, burst ≥ 32 is needed to plateau (~ 131 Mpps). The “knee” of each curve shifts right with load.
2. **Beyond the knee, larger burst gives no throughput.** At 25% rate the TX rate is flat from burst = 4 through burst = 256 within 0.5%. Same at 50% rate from burst = 8 on. So in any operating regime, only the smallest burst that hits the rate has throughput value.

If throughput were the only metric, this would be a negative result for tuning: just pick the largest plausible burst (e.g., default = 32) and forget about it. To capture the latency dimension directly, we instrumented `pktgen_send_pkts()` with `rte_rdtsc()` around the outer do-while loop that drains one app-level burst (DEFAULT_PKT_TX_BURST packets), accumulating per-lcore log-2 bucketed histograms during each sweep run. The TSC-based instrumentation adds ~ 50 cycles per burst (well below 1% overhead), so the measured throughput

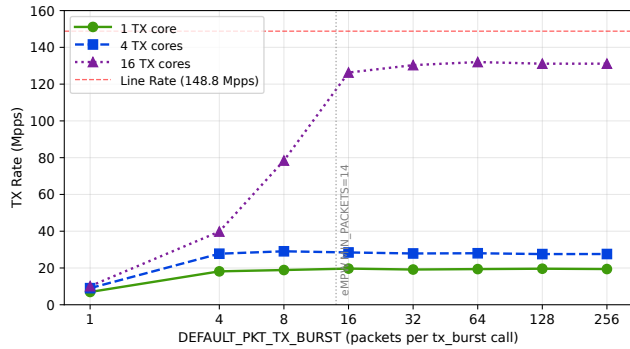


Figure 9: TX rate vs. `DEFAULT_PKT_TX_BURST` across three TX core counts. 1- and 4-core curves plateau by burst=4; the 16-core curve shows a $\sim 3.2\times$ jump between burst=8 and burst=16, crossing the `MLX5_EMPW_MIN_PACKETS=14` threshold that activates inline-eMPW. Beyond burst=16 the 16-core curve saturates near 130 Mpps.

is unaffected. Figure 10 (bottom) plots the resulting *measured* p99 latency per app-burst:

- **At 25% (rate-limited) and 50% loads**, the tail grows mildly with burst: p99 stays in the 0.7–1.4 μs range for burst ≤ 128 and roughly doubles to 2.8 μs at burst=256. The same burst that saturates the rate (e.g., burst=4 at 25%) already achieves a near-minimum tail; bigger bursts pay tail latency without gaining throughput.
- **At 100% load (saturation regime)**, the tail explodes: as soon as burst crosses the saturation knee at burst=16, p99 jumps from $\sim 1.4 \mu\text{s}$ to 22 μs (a $16\times$ step), then to 44 μs at burst=128 and beyond. This is the cost of running at the link limit: the NIC occasionally back-pressures, and the inner retry loop spins waiting for descriptors to drain. The *average* latency stays small (p50 < 3 μs) but a 1-in-100 slow burst is 15–30 $\times$ larger than at moderate load.
- **Burst=1 is uniformly bad on the latency axis too**, not just throughput: at every rate, burst=1 has p99 of 5.5 μs — worse than burst=4–128 at 25%/50% rate and matching the worst non-saturation case. Tiny bursts hit Pktgen’s per-call overhead path repeatedly without amortization.

Cross-comparison: what actually drives the tail latency?

The tail growth in Figure 10 (bottom) admits two competing explanations. **H1 (CPU cache pressure)**: larger bursts walk a bigger mbuf/header working set per call, evicting hot lines from L1/LLC, so per-packet CPU work grows. **H2 (NIC backpressure)**: as the application drives the link toward saturation, the NIC begins to back-pressure ($\sim$ cycle the inner retry loop in `pktgen_send_pkts()`), and / or its internal IOMMU caches (MTT, IOTLB) thrash.

To discriminate, we re-ran the same 24-point burst $\times$ rate sweep with all profilers enabled and cross-compared six metrics in Figure 11: measured p99 latency (rdtsc), CPU cycles

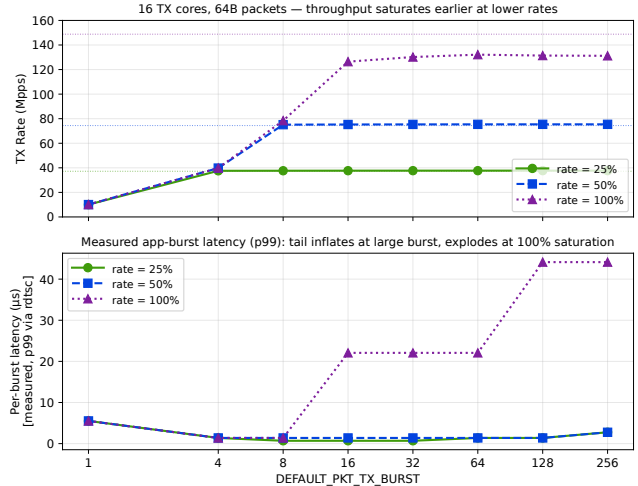


Figure 10: Rate-limited burst sweep at 16 TX cores with *measured* per-app-burst tail latency. **Top**: TX rate vs. burst for offered loads of 25/50/100% line rate; dotted horizontal lines mark the per-rate targets. The minimum burst that saturates the target shifts right with load (4 $\rightarrow$ 8 $\rightarrow$ 32). **Bottom**: p99 of the time spent in the outer do-while loop that dispatches one app-level burst, measured via rdtsc instrumentation in `pktgen_send_pkts()` (μs , log scale). At rate-limited loads the tail grows mildly with burst; at 100% saturation it jumps $16\times$ at burst=16 due to NIC back-pressure stalls.

per packet and LLC-load-misses per packet (test H1), and NIC-side PCIe avg latency, MTT L0 miss rate, and IOTLB miss rate (test H2).

H1 is refuted by the data. `cycles_per_pkt` is essentially flat from burst=4 onward at every rate (1356, 1352, 1352, 1357, 1352, 1344, 1345 cycles/pkt at rate 25% from burst 4 through 256 — variation <1%). LLC-load-misses per packet *decreases* with burst (0.0035 $\rightarrow$ 0.0019 across the same sweep at rate 25%): larger bursts amortize CPU cache fills across more packets, the opposite of what H1 predicts.

H2 is supported, with a clear tipping point at burst=16 under saturation. At 100% rate, the moment the sweep crosses burst=16 (where the throughput knee in Figure 10 sits), *all three* NIC-side metrics step up together: PCIe Avg Latency 455 $\rightarrow$ 690 ns (+52%), MTT L0 miss rate 1.71 $\rightarrow$ 2.88% (+68%), IOTLB miss rate 6.83% $\rightarrow$ 11.2% (+64%). These are exactly the inputs that would produce longer NIC-side DMA round trips, which feed back as the inner retry loop in `pktgen_send_pkts()` spinning for descriptors — precisely what the rdtsc histogram captures as a p99 jump from 1.4 μs to 22 μs ($16\times$). At the rate-limited operating points (25%, 50%) those same NIC counters stay nearly flat across the burst sweep, and the tail grows only mildly — roughly with burst (per-call work), not as a cache effect.

The net picture is that “burst size hurts latency at large

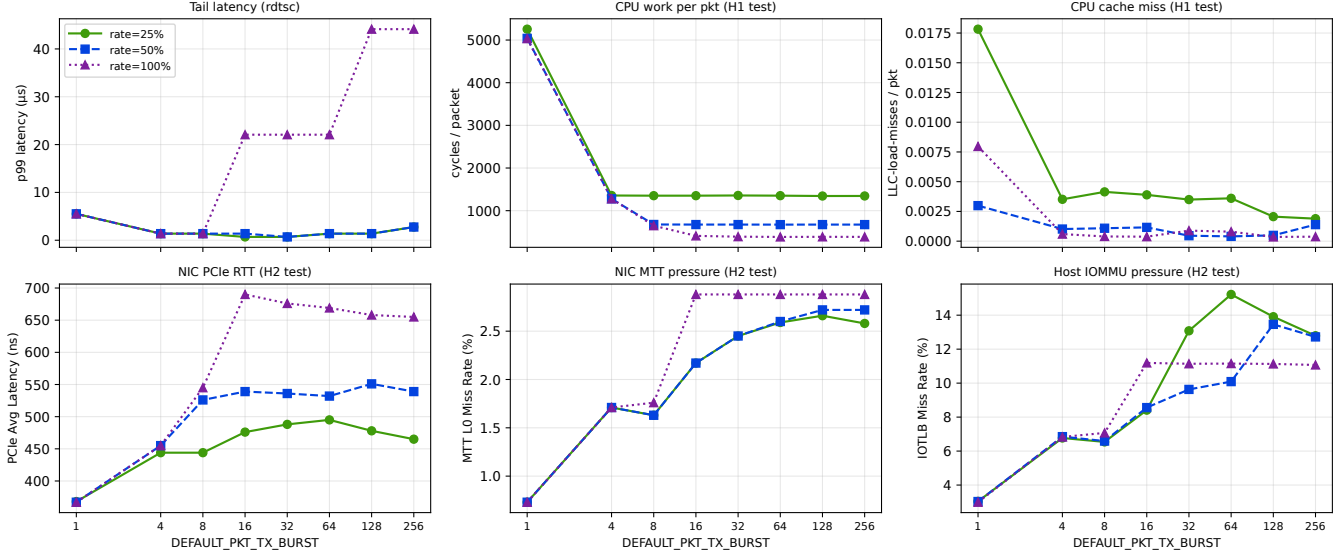


Figure 11: Cross-comparison at 16 TX cores, 64-byte packets, for offered loads of 25/50/100%. **Top row:** measured p99 tail latency (rdtsc) and CPU-side metrics that would correlate with H1 (cache pressure). cycles/pkt and LLC-load-miss/pkt do *not* grow with burst, so H1 is refuted. **Bottom row:** NIC- and IOMMU-side metrics that would correlate with H2 (NIC stall). At 100% rate, all three jump at burst = 16 in lockstep with the tail-latency explosion, supporting H2.

values” is two different mechanisms in disguise: **(a)** under load that does not saturate the NIC, larger burst simply puts more packets into a single `tx_burst` call, so the call takes proportionally longer — 4× from burst 64 to burst 256 in our measurement; **(b)** once the offered load reaches the link/NIC capacity, larger burst pushes the NIC into the back-pressure regime where MTT/IOTLB pressure causes occasional multi-microsecond stalls (the 22–44 μ s tail at 100% rate). Neither mechanism is CPU cache pressure.

Caveat: retry-loop specificity of the saturation tail. The 22–44 μ s tail at 100% rate is, strictly, a property of Pktgen’s TX path, not of `rte_eth_tx_burst()` in general. Pktgen wraps the API call in a do-while loop that spins until every requested packet has been accepted by the NIC SQ (see `tx_send_packets()` in `pktgen.c`). When the NIC back-pressures (SQ full because completion DMAs are slow under MTT/IOTLB pressure), this loop spins, and the rdtsc bracket we placed around it accumulates that spin time into the tail.

A more typical DPDK forwarding application (e.g., the L3FWD example) does *not* retry: it accepts the partial return from `rte_eth_tx_burst()` and `rte_pktmbuf_free()`s — or re-queues — the un-accepted mbufs, then moves on. In that programming model, the same NIC back-pressure manifests as *packet drops* (rising `tx_queue.q_errors / oerrors` counters), not as longer per-call latency. The underlying NIC-side cause is identical; only the surface metric differs. Our measurement should therefore be read as “*retry-loop apps will see this as a tail-latency wall; drop-tolerant apps will see*

this as a drop wall”; either way it is the operating point that an adaptive policy should learn to avoid.

Packet-size probe: where the burst dependency comes from. The 64-byte experiment in Figure 10 suggests that burst size matters because of eMPW activation. To verify this attribution, we re-ran the 16-core 100%-rate sweep with packet sizes {64, 256, 1500} B and compare in Figure 12.

- 64 B:** strong burst dependency. Throughput climbs from 10 Mpps (burst = 1) to 78 Mpps (burst = 8) to 127 Mpps (burst = 16) to a plateau of $\sim$ 131 Mpps. The cliff at burst = 14 matches `MLX5_EMPW_MIN_PACKETS`.
- 256 B:** almost flat from burst = 4 on (33.7 Mpps at burst = 4 vs. 35.3 Mpps at burst = 256: <5% delta). Per-packet payload is now large enough that the inline-eMPW data segment (`txq_inline_mpw` $\approx$ 268 B) holds at most one packet, so the multi-packet packing that 64-byte runs benefit from is unavailable. Bursting still amortizes per-call overhead, but the per-call overhead itself is a small fraction of the work for 256 B packets.
- 1500 B (MTU):** essentially independent of burst. Even burst = 1 reaches 7.7 Mpps (94% of the 8.22 Mpps line rate), and burst $\geq$ 16 saturates the link exactly. Each WQE carries a single packet via pointer mode (the packet far exceeds the inline budget), so the entire eMPW machinery is dormant.

Latency consequence: the strongest tuning motivation lives at 256 B and 1500 B. The throughput-only view of

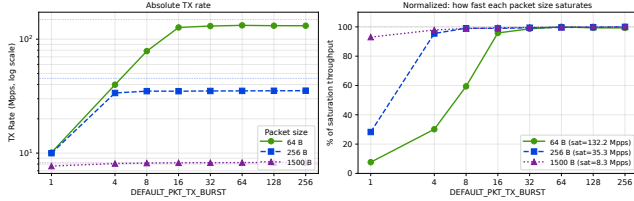


Figure 12: Burst sweep at 16 TX cores, 100% rate, across three packet sizes (64/256/1500 B). **Left:** absolute TX rate, log y-scale; dotted horizontal lines mark each packet size’s 100 GbE line rate. **Right:** same data normalized to each packet size’s saturation throughput. Smaller packets show a strong burst-size dependency driven by `MLX5_EMPW_MIN_PACKETS`; large packets are independent of burst because eMPW packing is structurally unavailable for them.

Figure 12 would suggest that for 256 B and 1500 B, burst size is essentially irrelevant (curves are flat). Once we re-measure the same sweep with our `rdtsc` app-burst latency instrumentation, the opposite story emerges: throughput stays flat, but p99 per-burst latency grows almost perfectly linearly with burst, all the way to hundreds of microseconds. Figure 13 shows the joint view.

The 1500 B row makes the point cleanest. TX rate ranges from 8.09 Mpps at burst = 4 to 8.24 Mpps at burst = 256 — a 1.8% span, well within run-to-run noise — so a throughput-only optimizer sees these eight burst values as interchangeable. The measured p99 latency over the same eight points goes 11 → 22 → 44 → 88 → 176 → 353 → 706 μ s, doubling at each step. The Pktgen default of burst = 32 sits at 88 μ s of p99 latency, 8 $\times$ worse than burst = 4 (11 μ s) at literally the same throughput. The 256 B row tells the same story at a smaller magnitude: TX rate flat at $\sim$ 35 Mpps from burst = 8 on, p99 latency 22 → 44 → 88 → 176 μ s from burst = 8 to 256 (8 $\times$).

Why this happens: for non-eMPW packet sizes (the inline-eMPW packing budget is $\sim$ 268 B, so 256 B and 1500 B essentially go through pointer mode), each `rte_eth_tx_burst()` call is processed packet-by-packet. The per-packet cost has a meaningful floor — the NIC issues a separate DMA-Read of the payload, the WQE is still constructed and a doorbell is rung. Per-call latency therefore scales as (burst) $\times$ (per-packet PMD/NIC work), exactly the linear growth visible in Figure 13. Throughput is decoupled from this because at 1500 B even burst = 1 already saturates the wire (94% of line rate); the NIC does not need batching to keep up with the link, so larger bursts simply hold the CPU longer in the dispatch loop with nothing to show for it.

The contrast with 64 B is sharp and illuminating. For 64 B the optimizer faces a real throughput vs. tail-latency trade-off across the eMPW threshold: burst = 8 gives 78 Mpps and 1.4 μ s p99, while burst $\geq$ 16 gives 126–132 Mpps and 22–44 μ s p99. Picking burst there is a workload-dependent decision. For 256 B and 1500 B there is no trade-off at all:

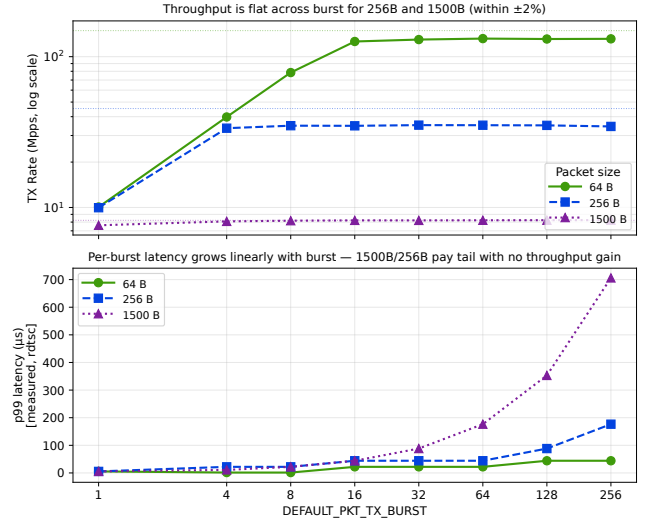


Figure 13: Same 24-point sweep as Figure 12 (16 TX cores, 100% rate, packet sizes 64/256/1500 B), now with the measured p99 per-burst latency overlaid. **Top:** TX rate (log y). For 256 B and 1500 B the curves are flat from burst = 4 onward — throughput is essentially the same whether burst is 4 or 256. **Bottom:** measured p99 dispatch latency (log y). For 1500 B, the same flat-throughput sweep pays 11 μ s at burst = 4 but 706 μ s at burst = 256 — a 64 $\times$ latency tax for no throughput benefit at all.

smaller burst is strictly better. The Pktgen default of 32 is calibrated for a 64 B-line-rate generator workload, and it is exactly wrong — by $\sim$ 8 $\times$ in p99 latency — for the much more common medium-to-large packet operating points.

Takeaway. `DEFAULT_PKT_TX_BURST` is a regime-dependent knob in three distinct senses: (1) the minimum-burst-to-saturate depends on offered load (a function of TX core count and rate, modulated by `MLX5_EMPW_MIN_PACKETS` = 14); (2) beyond that minimum, larger burst is a pure latency tax that earns nothing; (3) the entire dependency vanishes once the packet size grows past the inline-eMPW packing budget, because the PMD path that benefits from large bursts is itself a small-packet feature. A single static default (Pktgen’s 32) is therefore wrong in opposite directions: under light load or with large packets it sets the inter-call interval far higher than necessary, and under heavy load with small packets it remains workable but only because the eMPW threshold happens to lie below it. Picking the correct burst at runtime requires knowing the current offered rate, the dominant packet size, and the eMPW threshold of the underlying PMD — a multi-axis joint observation that is hard to encode as a fixed rule but natural for a learned policy fed live throughput / load telemetry.

References

- [1] Qizhe Cai, Shubham Chaudhary, Midhul Vuppalapati, Jaehyun Hwang, and Rachit Agarwal. Understanding host network stack overheads. In *Proceedings of the 2021 ACM SIGCOMM Conference*, pages 65–77, 2021.
- [2] Alireza Farshin, Amir Roozbeh, Gerald Q Maguire Jr, and Dejan Kostić. Reexamining direct cache access to optimize {I/O} intensive applications for multi-hundred-gigabit networks. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 673–689, 2020.
- [3] Intel Corporation. 3rd gen intel xeon scalable processor, codename ice lake, datasheet, volume two: Registers. Datasheet.
- [4] Intel Corporation. Intel xeon processor scalable family, datasheet, volume two: Registers. Datasheet.
- [5] Intel Corporation. Intel data direct i/o technology (intel ddio): A primer. Technical brief, February 2012. Revision 1.0.
- [6] Intel Corporation. Intel xeon processor scalable memory family uncore performance monitoring. Reference manual, 2017.
- [7] Intel Corporation. Utilizing the intel xeon processor scalable family iio performance monitoring events. <https://www.intel.com/content/www/us/en/developer/articles/technical/utilizing-the-intel-xeon-processor-scalable-family-iio-performance-monitoring-events.html>, 2019.
- [8] Intel Corporation. Intel virtualization technology for directed i/o. Architecture specification, March 2023. Revision 4.1.
- [9] Intel Support Community. Adjusting IIO_LLC_WAYS for increased ddio portion on intel xeon gold 6354. <https://community.intel.com/t5/Software-Tuning-Performance/Adjusting-IIO-LLC-WAYS-for-Increased-DDIO-Portion-on-Intel-Xeon/m-p/1640393>, 2024.